



AU : 2025– 2026

JAVA

TP-Cours (Suite)

Programmation Orientée Objet

Section 1 — Architecture du Projet Zoo

Tout au long de ce TP, vous allez construire pas à pas un **système de gestion d'un zoo**. Chaque partie introduit un nouveau concept POO en s'appuyant sur ce projet commun. À la fin, vous disposerez d'une application Java complète et bien structurée.

1.1 Vue d'ensemble

Structure des fichiers Java à créer

Structure du projet

```
zoo/
├─ Animal.java      ← Classe abstraite de base
├─ Nourrissable.java ← Interface
├─ Lion.java        ← Sous-classe (Mammifère)
├─ Perroquet.java   ← Sous-classe (Oiseau)
├─ Requin.java      ← Sous-classe (Poisson)
├─ Enclos.java      ← Classe utilitaire
└─ Main.java        ← Point d'entrée du programme
```

1.2 Diagramme de classes simplifié

Diagramme de classes

```
<<abstract>>
Animal
┌───────────┴───────────┐
- nom : String
- age : int
- poids : double
└───────────┬───────────┘
+ getNom() : String
+ setNom(String)
+ faireDuBruit() *      ← abstract
+ toString() : String
    |
    ┌──┴──┐
    |     |
Lion  Perroquet  Requin
┌───┴──┐ ┌───┴──┐ ┌───┴──┐
race   couleur  espece
rugir() parler() nager()
```

Section 2 — Classe & Objet

Rappel théorique

Une classe est un modèle (blueprint) qui décrit les attributs et comportements d'un type d'objet.
Un objet est une instance concrète créée à partir de ce modèle avec le mot-clé `new`.

2.1 Créer la classe `Animal` de base

Commençons par une classe simple (sans abstraction pour l'instant). Créez le fichier **`Animal.java`** dans votre projet :

```
Java
public class Animal {

    // — Attributs —————
    String nom;
    int age;
    double poids;

    // — Constructeur —————
    public Animal(String nom, int age, double poids) {
        this.nom = nom;
        this.age = age;
        this.poids = poids;
    }

    // — Méthode —————
    public void afficherInfos() {
        System.out.println("Animal : " + nom +
                           " | Age : " + age +
                           " | Poids : " + poids + " kg");
    }
}
```

2.2 Créer et utiliser des objets dans `Main.java`

```
        a2.afficherInfos();
    }
}
```

Résultat attendu dans la console

Animal : Simba | Age : 5 | Poids : 190.5 kg Animal : Coco | Age : 3 | Poids : 0.4 kg

Exercice 1 — Classe & Objet**Consignes :**

1. Créez la classe Animal telle que présentée ci-dessus.
2. Ajoutez un troisième attribut String espece (ex: "Félin", "Oiseau", "Poisson").
3. Modifiez afficherInfos() pour inclure l'espèce dans l'affichage.
4. Dans Main.java, instanciez 3 animaux différents et affichez leurs informations.

Solution

// Animal.java – Solution exercice 1

```
public class Animal {
    String nom;
    int age;
    double poids;
    String espece; // ← nouveau attribut

    public Animal(String nom, int age, double poids, String espece) {
        this.nom = nom;
        this.age = age;
        this.poids = poids;
        this.espece = espece;
    }

    public void afficherInfos() {
        System.out.println("[ " + espece + " ] " + nom +
            " | Age : " + age + " ans" +
            " | " + poids + " kg");
    }
}
```

Section 3 — Encapsulation

Rappel théorique

L'encapsulation consiste à rendre les attributs privés (private) et à contrôler leur accès via des méthodes publiques : getters (lecture) et setters (écriture avec validation).

3.1 Modificateurs d'accès

Modificateur	Même classe	Package	Partout
private	+	-	-
(package)	+	+	-
protected	+	+	Sous-classes
public	+	+	+

3.2 Refactoriser la classe Animal avec encapsulation

Modifiez Animal.java pour passer tous les attributs en private et ajouter les getters/setters :

Java

```
public class Animal {

    // — Attributs PRIVÉS —
    private String nom;
    private int    age;
    private double poids;
    private String espece;

    // — Constructeur —
    public Animal(String nom, int age, double poids, String espece) {
        this.nom    = nom;
        setAge(age);    // utilise le setter pour valider
        setPoids(poids);
        this.espece = espece;
    }

    // — Getters —
    public String getNom()    { return nom; }
    public int    getAge()    { return age; }
    public double getPoids()  { return poids; }
    public String getEspece() { return espece; }

    // — Setters avec validation —
```

```
public void setNom(String nom) {
    if (nom != null && !nom.isEmpty())
        this.nom = nom;
}

public void setAge(int age) {
    if (age >= 0 && age <= 100)
        this.age = age;
    else
        System.out.println("Erreur : age invalide (" + age + ")");
}

public void setPoids(double poids) {
    if (poids > 0)
        this.poids = poids;
}

// — Méthode d'affichage —————
@Override
public String toString() {
    return "[" + espece + "] " + nom +
        " | Age : " + age + " ans" +
        " | " + poids + " kg";
}
}
```

Exercice 2 — Encapsulation

Consignes :

5. Refactorisez Animal.java avec les attributs private et les getters/setters ci-dessus.
6. Dans Main.java, essayez d'accéder directement à a1.nom (vous obtiendrez une erreur de compilation — c'est voulu !)
7. Utilisez a1.setNom("Nala") et a1.getNom() pour modifier et lire la valeur.
8. Testez le setter setAge() avec une valeur négative (-5) et observez le message d'erreur.

Section 4 — Héritage

Rappel théorique

L'héritage permet à une classe (sous-classe) de réutiliser et d'étendre les attributs et méthodes d'une autre classe (super-classe) grâce au mot-clé `extends`. Le constructeur parent est appelé avec `super()`.

4.1 Créer la classe Lion

```
Java
public class Lion extends Animal {

    // — Attribut spécifique au Lion —
    private String race;    // ex: "Berbère", "Africain"

    // — Constructeur —
    public Lion(String nom, int age, double poids, String race) {
        // Appel du constructeur parent avec super()
        super(nom, age, poids, "Félin");
        this.race = race;
    }

    // — Méthode propre au Lion —
    public void rugir() {
        System.out.println(getNom() + " rugit : ROARRR ! ");
    }

    // — Getter —
    public String getRace() { return race; }

    @Override
    public String toString() {
        return super.toString() + " | Race : " + race;
    }
}
```

4.2 Créer la classe Perroquet

```
Java
public class Perroquet extends Animal {

    private String couleurPlumage;
    private String motAppris;    // mot que le perroquet répète

    public Perroquet(String nom, int age, double poids,
        String couleur, String mot) {
```

```

        super(nom, age, poids, "Oiseau");
        this.couleurPlumage = couleur;
        this.motAppris      = mot;
    }

    public void parler() {
        System.out.println(getNom() + " dit : " + motAppris + " ! ");
    }

    @Override
    public String toString() {
        return super.toString() +
            " | Plumage : " + couleurPlumage;
    }
}

```

4.3 Créer la classe Requin

```

Java
public class Requin extends Animal {

    private String espece;      // ex: "Blanc", "Marteau"
    private double longueur;    // en mètres

    public Requin(String nom, int age, double poids,
                   String espece, double longueur) {
        super(nom, age, poids, "Poisson");
        this.espece = espece;
        this.longueur = longueur;
    }

    public void nager() {
        System.out.println(getNom() + " nage silencieusement... ");
    }

    @Override
    public String toString() {
        return super.toString() +
            " | " + espece + " " + longueur + "m";
    }
}

```

Exercice 3 — Héritage — Test dans Main.java

Consignes :

9. Créez les 3 classes (Lion, Perroquet, Requin) dans des fichiers séparés.

- 10.** Dans Main.java, instanciez un Lion ("Simba", 5, 190.0, "Africain").
- 11.** Instanciez un Perroquet ("Coco", 3, 0.4, "Vert", "Bonjour").
- 12.** Instanciez un Requin ("Jaws", 10, 680.0, "Grand Blanc", 5.5).
- 13.** Appelez System.out.println() sur chaque objet (cela appelle toString()).
- 14.** Appelez les méthodes spécifiques : rugir(), parler(), nager().

Section 5 — Polymorphisme

Rappel théorique

@Override permet à une sous-classe de redéfinir une méthode héritée. Le polymorphisme de type permet d'utiliser une variable de type parent pour référencer un objet fils. La méthode exécutée est celle de la classe réelle (liaison dynamique).

5.1 Méthode faireDuBruit() dans chaque sous-classe

Ajoutez la méthode faireDuBruit() à chaque classe (avec @Override) :

```
Java
// Dans Animal.java – ajoutez la méthode de base :
public void faireDuBruit() {
    System.out.println(getNom() + " fait un bruit générique...");
}

// Dans Lion.java – redéfinition avec @Override :
@Override
public void faireDuBruit() {
    System.out.println(getNom() + " RUGIT ! ROARRR !");
}

// Dans Perroquet.java :
@Override
public void faireDuBruit() {
    System.out.println(getNom() + " crie : " + motAppris + " !");
}

// Dans Requin.java :
@Override
public void faireDuBruit() {
    System.out.println(getNom() + " ... (silence) ...");
}
```

5.2 Polymorphisme de type — le tableau Animal[]

Voici la puissance du polymorphisme : on peut stocker des objets de types différents dans un tableau du type parent !

```
Java
// Main.java – Polymorphisme en action
public class Main {
    public static void main(String[] args) {

        // Tableau de type Animal contenant des sous-types
    }
}
```

```
Animal[] zoo = {
    new Lion("Simba", 5, 190.0, "Africain"),
    new Perroquet("Coco", 3, 0.4, "Vert", "Coucou"),
    new Requin("Jaws", 10, 680.0, "Grand Blanc", 5.5),
    new Lion("Nala", 4, 140.0, "Berbère"),
};

// Appel uniforme de faireDuBruit() sur tous les animaux
System.out.println("=== Le zoo s'anime ! ===");
for (Animal a : zoo) {
    System.out.print("- ");
    a.faireDuBruit();    // liaison dynamique !
}

// instanceof pour identifier le type réel
for (Animal a : zoo) {
    if (a instanceof Lion) {
        Lion l = (Lion) a;    // cast
        l.rugir();
    }
}
}
```

Liaison dynamique

Quand on appelle `a.faireDuBruit()` sur une variable de type `Animal`, Java exécute la version de la méthode correspondant au **TYPE RÉEL** de l'objet (`Lion`, `Perroquet` ou `Requin`), pas le type de la variable. C'est la **liaison dynamique** (*late binding*).

Exercice 4 — Polymorphisme — Méthode utilitaire

Consignes :

Créez une méthode statique dans `Main.java` : `static void nourrirAnimal(Animal a)`

15. La méthode affiche "Nourrissage de " + `a.getNom()`
16. Puis appelle `a.faireDuBruit()`
17. Appelez `nourrirAnimal()` sur chaque animal du tableau `zoo[]`
18. Observez que la bonne version de `faireDuBruit()` est appelée automatiquement !

Section 6 — Abstraction

Rappel théorique

Une classe abstraite (abstract class) ne peut pas être instanciée directement. Elle peut contenir des méthodes abstraites (sans corps) que les sous-classes DOIVENT implémenter. Une interface définit un contrat pur (toutes les méthodes sont implicitement abstraites et publiques).

6.1 Rendre Animal abstraite

Modifiez la déclaration de `Animal.java` — ajoutez le mot-clé **abstract** et rendez `faireDuBruit()` abstraite :

Java

```
// Animal.java – version abstraite finale
public abstract class Animal {

    private String nom;
    private int    age;
    private double poids;
    private String espece;

    public Animal(String nom, int age, double poids, String espece) {
        this.nom    = nom;
        setAge(age);
        setPoids(poids);
        this.espece = espece;
    }

    // — Méthode ABSTRAITE → sous-classes obligées d'implémenter —
    public abstract void faireDuBruit();

    // — Méthodes concrètes —
    public String getNom()    { return nom; }
    public int    getAge()    { return age; }
    public double getPoids() { return poids; }
    public String getEspece() { return espece; }

    public void setNom(String nom) {
        if (nom != null && !nom.isEmpty()) this.nom = nom;
    }
    public void setAge(int age) {
        if (age >= 0 && age <= 100) this.age = age;
    }
    public void setPoids(double p) {
        if (p > 0) this.poids = p;
    }
}
```

```

@Override
public String toString() {
    return "[" + espece + "]" + nom +
        " | Age : " + age + " | " + poids + " kg";
}
}

```

6.2 Créer l'interface Nourrissable

Une interface définit des comportements que plusieurs classes (même non liées) peuvent implémenter :

```

Java
// Nourrissable.java
public interface Nourrissable {

    // Méthodes abstraites (public abstract implicite)
    void manger(String nourriture);
    double getRationJournaliere(); // en kg

    // Méthode par défaut (Java 8+)
    default void afficherRegime() {
        System.out.println("Ration journalière : " +
            getRationJournaliere() + " kg");
    }
}

```

6.3 Implémenter l'interface dans Lion

```

Java
// Lion.java – version finale avec interface
public class Lion extends Animal implements Nourrissable {

    private String race;

    public Lion(String nom, int age, double poids, String race) {
        super(nom, age, poids, "Félin");
        this.race = race;
    }

    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " RUGIT ! ROARRR ! ");
    }

    @Override
    public void manger(String nourriture) {
        System.out.println(getNom() + " dévore : " + nourriture + " ");
    }
}

```

```
}

@Override
public double getRationJournaliere() {
    return getPoids() * 0.04; // 4% du poids corporel
}

public void rugir()          { faireDuBruit(); }
public String getRace()      { return race; }

@Override
public String toString() {
    return super.toString() + " | Race : " + race;
}
}
```

Exercice 5 — Abstraction & Interface

Consignes :

19. Rendez `Animal` abstraite et `faireDuBruit()` abstraite.
20. Créez l'interface `Nourrissable.java`.
21. Faites implémenter `Nourrissable` par `Lion`, `Perroquet` et `Requin` (rationjournalière adaptée à chaque espèce).
22. Essayez `new Animal(...)` — vous obtiendrez une erreur de compilation. Expliquez pourquoi.
23. Dans `Main.java`, appelez `afficherRegime()` sur chaque animal (hérité de l'interface !).

Section 7 — Classe Enclos (Composition)

La **composition** est une autre relation entre classes : une classe peut contenir des objets d'une autre classe (ici, un Enclos contient une liste d'Animaux).

Java

```
import java.util.ArrayList;
import java.util.List;

public class Enclos {

    private String    nom;
    private String    type;        // ex: "Savane", "Aquarium"
    private int        capaciteMax;
    private List<Animal> animaux = new ArrayList<>();

    public Enclos(String nom, String type, int capaciteMax) {
        this.nom        = nom;
        this.type        = type;
        this.capaciteMax = capaciteMax;
    }

    // Ajouter un animal à l'enclos
    public boolean ajouterAnimal(Animal a) {
        if (animaux.size() < capaciteMax) {
            animaux.add(a);
            System.out.println(a.getNom() + " ajouté à " + nom);
            return true;
        }
        System.out.println("Enclos " + nom + " est plein !");
        return false;
    }

    // Faire faire du bruit à tous les animaux (polymorphisme !)
    public void reveillerAnimaux() {
        System.out.println("\n=== Réveil de " + nom + " ===");
        for (Animal a : animaux) {
            a.faireDuBruit();        // polymorphisme en action
        }
    }

    public void afficherAnimaux() {
        System.out.println("Enclos [" + type + "] " + nom +
            " (" + animaux.size() + "/" + capaciteMax + ") :");
        animaux.forEach(a -> System.out.println("  - " + a));
    }
}
```

7.1 Main.java — Assemblage final du Zoo

Java

```
public class Main {
    public static void main(String[] args) {

        // Créer les enclos
        Enclos savane = new Enclos("Savane Africaine", "Mammifères", 4);
        Enclos aquarium = new Enclos("Grand Aquarium", "Poissons", 3);
        Enclos voliere = new Enclos("Volière Tropicale", "Oiseaux", 5);

        // Créer les animaux
        Lion simba = new Lion("Simba", 5, 190.0, "Africain");
        Lion nala = new Lion("Nala", 4, 135.0, "Berbère");
        Perroquet coco = new Perroquet("Coco", 3, 0.4, "Vert", "Bonjour");
        Perroquet rio = new Perroquet("Rio", 2, 0.5, "Rouge", "Ola !");
        Requin jaws = new Requin("Jaws", 10, 680.0, "Grand Blanc", 5.5);

        // Placer les animaux
        savane.ajouterAnimal(simba);
        savane.ajouterAnimal(nala);
        aquarium.ajouterAnimal(jaws);
        voliere.ajouterAnimal(coco);
        voliere.ajouterAnimal(rio);

        // Afficher le zoo
        System.out.println("\n=== BIENVENUE AU ZOO ===\n");
        savane.afficherAnimaux();
        aquarium.afficherAnimaux();
        voliere.afficherAnimaux();

        // Réveiller tous les animaux (polymorphisme !)
        savane.reveillerAnimaux();
        voliere.reveillerAnimaux();

        // Nourrir tous les animaux (interface Nourrissable)
        Animal[] tous = { simba, nala, coco, rio, jaws };
        System.out.println("\n=== Nourrissage du soir ===");
        for (Animal a : tous) {
            if (a instanceof Nourrissable n) {
                n.afficherRegime();
            }
        }
    }
}
```

Section 8 — Récapitulatif & Tableau de Synthèse

Voici un résumé de tous les concepts POO appliqués dans ce TP :

Concept	Mot-clé Java	Rôle	Exemple dans le TP
Classe & Objet	class / new	Définir un type et créer des instances	Animal, Lion, Enclos
Encapsulation	private / get / set	Protéger les données avec validation	Animal.setAge() vérifie ≥ 0
Héritage	extends / super()	Réutiliser et spécialiser une classe	Lion extends Animal
Polymorphisme	@Override	Comportement adapté selon le type réel	zoo[i].faireDuBruit()
Classe abstraite	abstract class	Interdire l'instanciation directe + contrat	Animal ne peut pas être instancié
Interface	interface / implements	Contrat multi-héritage de comportements	Lion implements Nourrissable
Composition	List<T> / ArrayList	Contenir des objets d'autres classes	Enclos contient Animal[]

Différences importantes à retenir

Classe Abstraite	Interface
Peut avoir des méthodes concrètes	Méthodes concrètes via default seulement
Peut avoir des attributs d'instance	Que des constantes (static final)
Une classe hérite d'une seule abstraite	Une classe peut implémenter plusieurs interfaces
Utilisez quand le lien 'est un' est fort	Utilisez pour définir un comportement partagé